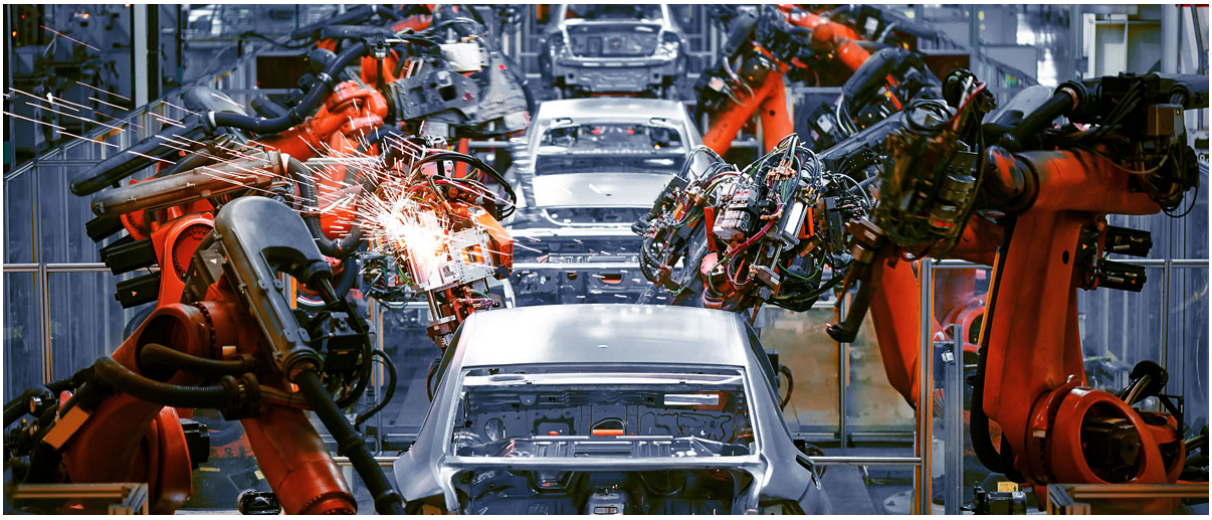
Chapter 6: Automating Feature-Level Exploration

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to design and implement reusable functions that iterate through DataFrame columns and compute appropriate statistics based on data type <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to use branching logic to differentiate between numeric and categorical processing paths within automated exploration functions <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to write dynamic, error-resistant code that adapts to datasets of varying sizes and column compositions without manual modification <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to package automation functions in external Python modules for reuse across multiple projects

6.1Introduction



You already know the univariate statistics and visualizations needed during the Data Understanding phase. Writing them in Python is an important first step, but you can go further by automating the entire workflow. The goal is reusable code that quickly summarizes any dataset, so you spend less time on repetitive analysis and more time interpreting what the output tells you about the data.

Of course, every dataset has unique quirks that you can't fully automate. But many tasks repeat across projects, especially when they depend on the data type of each column. For example, earlier you checked whether each column in the insurance dataset was numeric:

```
print('age: ' + str(pd.api.types.is_numeric_dtype(df.age)))
print('sex: ' + str(pd.api.types.is_numeric_dtype(df.sex)))
print('bmi: ' + str(pd.api.types.is_numeric_dtype(df.bmi)))
print('children: ' + str(pd.api.types.is_numeric_dtype(df.children)))
print('smoker: ' + str(pd.api.types.is_numeric_dtype(df.smoker)))
print('region: ' + str(pd.api.types.is_numeric_dtype(df.region)))
print('charges: ' + str(pd.api.types.is_numeric_dtype(df.charges)))
```

That approach works, but it doesn't scale. The repeated pattern is the clue: when you see yourself copying and pasting the same code with only a small change (like a different column name), it's time to automate. One simple improvement is to loop through the columns:

```
# Print them using a loop
for col in df.columns:
  print(str(col) + ': ' + str(pd.api.types.is_numeric_dtype(df[col])))


# Output:
# age: True
# sex: False
# bmi: True
# children: True
# smoker: False
# region: False
# charges: True
```

Loops reduce repetition, but they are only the beginning. This chapter wraps these ideas into reusable functions that adapt to different datasets and handle unexpected input gracefully. By the end, a single function call will summarize any dataset and surface the patterns that drive data preparation: skewness, outliers, missing values, high-cardinality categories, columns

worth dropping. That diagnostic output drives every cleaning decision in the next chapter.

Images in this section were created using DALL·E from OpenAI.

6.2What Is Automation?

This video can be viewed online.

Good data science code does three things well. It reduces the manual work you do repeatedly, it adapts when the data changes, and it handles messy input without crashing. Those three qualities have names:

1. **Automated**
2. **Dynamic**
3. **Error-resistant**

These three ideas are related, but not identical: **automation** reduces repetition, **dynamic code** adapts to new datasets without manual rewrites, and **error-resistant code** keeps working even when inputs are messy or incomplete.

Here is a running example to keep things concrete: imagine you receive a new dataset every week with 200+ columns. Manually checking data types, missing values, and distributions for each one is slow and error-prone. Automation means writing that logic once and reusing it on every dataset that follows.

For simplicity, I will often use *automation* as shorthand for all three. The concepts differ, but in practice they tend to show up together.

What Does Automation Mean?

automation The process of replicating human effort and decision-making in programming code. refers to reducing human effort by turning repeated steps into code that runs the same way every time. In practice, automation usually combines iteration (loops), decision logic (if/else), and consistent outputs. For example, instead of manually typing code for each column in a dataset to determine its data type, you can write a loop that checks all columns automatically. This approach eliminates redundant code and minimizes manual intervention.

Some automation simply saves time. Other automation includes decision-making logic: choosing a different analysis path for numeric versus categorical data, for instance. That second kind becomes especially important when you begin building models.

What Is Dynamic Code?

Code that is dynamic In software, code that will work uninterrupted regardless of the amount or type of inputs that are provided. adapts to varying inputs and conditions without requiring manual adjustments. Dynamic processes ensure that your code works even when the dataset changes in:

1. **Input Data:** Whether the input dataset has different values or formats.
2. **Dataset Structure:** The number of columns, rows, or other structural features.

3. **Data Types:** Whether columns contain numeric, categorical, or mixed data.

For instance, a dynamic function designed to analyze datasets should handle small datasets as effectively as large ones, regardless of how many columns exist or what types they contain. In our weekly “200-column dataset” example, dynamic code means you do not have to rewrite your analysis when a new column is added, removed, renamed, or stored with a different data type.

Dynamic code travels well. You write it once, and it works on the next project without rewriting.

What Does Error-Resistant Mean?

Finally, error-resistant In the context of software, this refers to code that will work uninterrupted even if the user attempts to submit invalid inputs either by (1) specifying a better form for the inputs or (2) adapting or modifying the inputs to an acceptable form. code anticipates and handles issues that may arise during execution. In real datasets, common problems include missing columns, unexpected data types, empty datasets, and columns with all missing values. You can make your code error-resistant by:

1. **Validating Inputs:** Ensuring that the data meets required conditions before processing begins (for example, confirming an input is a DataFrame and contains expected columns).
2. **Adapting Inputs:** Transforming inputs into acceptable forms when possible (for example, handling missing values by imputing them, skipping invalid columns, or excluding affected rows).

Error resistance minimizes crashes and unexpected behavior, making your code more reliable and user-friendly. In our weekly dataset example, error-resistant code means your analysis still runs even if the dataset arrives with a missing column, a misspelled label, or a column that is unexpectedly stored as text instead of numbers.

With those three ideas in place, the next section puts them into practice: you will build Python functions that automatically analyze every column in a dataset.

6.3Automating Statistics

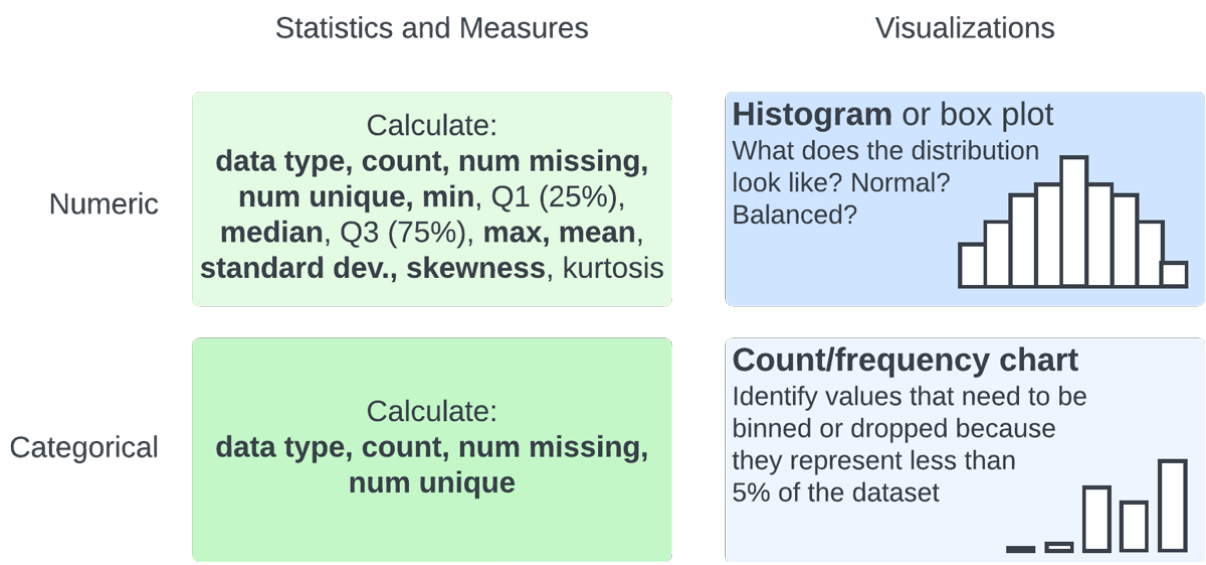


Figure 6.1: Summary of Univariate Statistics and Visualizations to Automate

The image above summarizes the list of statistics, measures, and visualizations that we typically want to create (the minimum in bold) as we explore a dataset. But where should we begin this process of automating stats and charts?

In the prior section, you learned that strong >automated (reduces repetition), **dynamic** (works across datasets), and **error-resistant** (fails gracefully when the data is messy). In this section, you'll apply those ideas by building a function that generates univariate statistics for **every** column in a DataFrame.

We'll start small, test early, and improve the function step-by-step. This is what real automation looks like: build a reliable base, then extend it with careful branching, clean outputs, and reusable packaging.

A Repeatable Automation Pattern

Automating the process of generating univariate statistics improves efficiency and consistency. If you're unsure where to start, use this general pattern. With practice, it will become second nature:

1. Define the automation function.
2. Import necessary Python packages.
3. Create variables for processing.
4. Define the iteration (e.g., a loop).
5. Perform processing for each iteration.
6. Define the decision criterion.
7. Perform processing based on the decision structure.
8. Synthesize and return the results.

Design Checklist (Before You Code)
Before writing the full function, decide what "success" looks like. At minimum: (1) input is a DataFrame, (2) output is a clean summary table, and (3) the function works even when columns are messy (missing values, unexpected dtypes, etc.).

A. Define the Function Interface

Step 1: Define the Automation Function

The first step is to define the function. This includes deciding on its input parameter(s) and output. Ask yourself:

1. What input data is necessary for the function to perform its task?
2. What should the function return?

In this case, we want a function that automates univariate statistics for all columns in a DataFrame. Thus, our input will be the entire DataFrame (**df**). Here's an initial implementation:

```
# Step 1: Define the Automation Function

def unistats(df):
  return df
```

Although simple, it's crucial to choose input parameters wisely. When designing functions, balance two criteria:

1. Minimize the required input to what's strictly necessary.
   - For example, don't require a Pandas Series if a basic Python list would work.
2. Choose input types that minimize additional preprocessing.
   - For example, pass a Pandas Series if you need row labels or convenient methods like .nunique().

Notice how these criteria can conflict. In this context, we want a function that can analyze **all** features automatically, so passing the full DataFrame (**df**) is the best choice.

Step 2: Import Python Packages

A function should be self-contained, meaning it cannot assume that the required packages are already available in memory. Therefore, explicitly import all packages needed for the function.

For our unistats function, we'll use Pandas to manage the DataFrame and calculate statistics:

```
def unistats(df):

  # Step 2: Import necessary packages
  import pandas as pd

  return df
```

Step 3: Create Variables for Processing

Next, consider the structure of the output. Will the function return a simple value (e.g., int, float, str, bool) or a collection (e.g., list, dict, DataFrame)? If it's a collection, initialize it before the loop begins.

For unistats, the output is a summary table containing univariate statistics for each column. We initialize it as an empty DataFrame:

```
def unistats(df):
  import pandas as pd
```

```
  # Step 3: Create a variable for processing
  output_df = pd.DataFrame()

  return output_df
```

B. Iterate and Compute Baseline Stats

Step 4: Define the Iteration

Data science tasks often involve applying repeated processes to columns or rows. For unistats, we need to iterate through each column of the DataFrame:

```
def unistats(df):
  import pandas as pd

  output_df = pd.DataFrame(columns=['Type'])

  # Step 4: Define the iteration
  for col in df.columns:
    pass  # Temporary placeholder

  return output_df
```

This loop enables us to process each column individually.

Step 5: Perform Processing for Every Iteration

Some operations apply to all columns, regardless of whether they contain numeric or categorical data. For instance:

- Counting non-missing values.
- Counting unique values.
- Identifying the data type.

These operations form the base of our unistats function. Update it as follows:

```
def unistats(df):
  import pandas as pd

  output_df = pd.DataFrame(columns=['Count', 'Unique', 'Type'])

  for col in df.columns:
    count = df[col].count()       # non-missing count
    unique = df[col].nunique()    # unique values
    dtype = str(df[col].dtype)    # dtype as a readable string
    output_df.loc[col] = [count, unique, dtype]

  return output_df
```

Now it’s time to test. Don’t wait until you think you’ve finished to test your code—test early and often. Below is a quick test call:

```
# Test out the function:
import pandas as pd

df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
unistats(df)

# Output:
#          Count Unique     Type
# age       1338     47    int64
# sex       1338      2   object
# bmi       1338    548  float64
# children  1338      6    int64
# smoker    1338      2   object
# region    1338      4   object
# charges   1338   1337  float64
```

Each time through the loop, a new row is added to output_df. You can think of it as “rotating” the DataFrame: columns in df become row labels in output_df. This is a common and readable summary format.

If you prefer the original orientation (columns stay as columns), you can build lists inside the loop and create the DataFrame after the loop finishes. Here’s one way to do it (optional):

```
def unistats(df):
  import pandas as pd

  counts = []
  uniques = []
  dtypes = []

  for col in df.columns:
    counts.append(df[col].count())
    uniques.append(df[col].nunique())
    dtypes.append(str(df[col].dtype))

  output_df = pd.DataFrame(
    [counts, uniques, dtypes],
    columns=df.columns,
    index=['Count', 'Unique', 'Type']
  )

  return output_df
```

```
# Test out the function:
import pandas as pd

df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
unistats(df)

# Output:
#           age     sex      bmi children  smoker  region  charges
# Count    1338    1338     1338     1338    1338    1338     1338
# Unique     47       2      548        6       2       4     1337
# Type    int64  object  float64    int64  object  object  float64
```

Which do you prefer? Use either format for the rest of this tutorial. For consistency, the rest of the examples will use the first format (columns become rows).

C. Branch by Data Type

Step 6: Define the Decision Criterion

To perform operations specific to numeric or categorical data, we need a decision criterion. Pandas provides pd.api.types.is_numeric_dtype() to

identify numeric columns. A practical and consistent pattern is to evaluate the Series itself:

```
def unistats(df):
  import pandas as pd

  output_df = pd.DataFrame(columns=['Count', 'Unique', 'Type'])

  for col in df.columns:
    count = df[col].count()
    unique = df[col].nunique()
    dtype = str(df[col].dtype)
    # Decision criterion: is the column numeric?
    if pd.api.types.is_numeric_dtype(df[col]):
      print(&quot;Testing: &quot; + col + &quot; is numeric&quot;)

    output_df.loc[col] = [count, unique, dtype]

  return output_df
```

Test it:

```
unistats(df)


# Output:
# Testing: age is numeric
# Testing: bmi is numeric
# Testing: children is numeric
# Testing: charges is numeric
```

Notice that the print statements execute from inside the function, and the DataFrame is still returned when the function finishes.

D. Add Branch-Specific Processing and Fix a Common Logical Bug

Step 7: Perform Processing in Each Branch

Now that we can control the flow based on data type, we can add additional processing for numeric columns. The key idea is: numeric columns get numeric statistics; categorical columns do not.

In the next two sections, you'll build standalone visualization functions and then integrate them with unistats(). For now, keep it focused: baseline stats for every column, plus common numeric stats for numeric columns.

```
def unistats(df):
  import pandas as pd

  output_df = pd.DataFrame(columns=[
    'Count', 'Unique', 'Type',
    'Min', 'Max', '25%', '50%', '75%',
    'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
  ])

  for col in df.columns:
    count = df[col].count()
    unique = df[col].nunique()
    dtype = str(df[col].dtype)
    # Initialize branch-specific values to placeholders each iteration
    min_val = '-'
    max_val = '-'
    q1 = '-'
    q2 = '-'
    q3 = '-'
    mean_val = '-'
    median_val = '-'
    mode_val = '-'
    std_val = '-'
    skew_val = '-'
    kurt_val = '-'

    if pd.api.types.is_numeric_dtype(df[col]):
      min_val = round(df[col].min(), 2)
      max_val = round(df[col].max(), 2)
      q1 = round(df[col].quantile(0.25), 2)
      q2 = round(df[col].quantile(0.50), 2)
      q3 = round(df[col].quantile(0.75), 2)
      mean_val = round(df[col].mean(), 2)
      median_val = round(df[col].median(), 2)
      # Mode can return multiple values; we'll take the first as a simple default
      mode_series = df[col].mode()
      mode_val = round(mode_series.values[0], 2) if len(mode_series) &gt; 0 else
'-'

      std_val = round(df[col].std(), 2)
      skew_val = round(df[col].skew(), 2)
      kurt_val = round(df[col].kurt(), 2)

    output_df.loc[col] = (
      count, unique, dtype,
```

```
            min_val, max_val, q1, q2, q3,
            mean_val, median_val, mode_val, std_val, skew_val, kurt_val
          )

        return output_df



      import pandas as pd

      df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
      unistats(df)

      # Output:
      #          Count Unique     Type      Min       Max     25%     50%      75%
Mean  Median     Mode       Std  Skew  Kurt
      # age       1338     47    int64       18        64    27.0    39.0     51.0
39.21    39.0       18     14.05  0.06 -1.25
      # sex       1338      2   object        -         -       -       -        -
-       -        -         -     -     -
      # bmi       1338    548  float64    15.96     53.13    26.3    30.4    34.69
30.66    30.4     32.3       6.1  0.28 -0.05
      # children  1338      6    int64        0         5     0.0     1.0      2.0
1.09     1.0        0      1.21  0.94   0.2
      # smoker    1338      2   object        -         -       -       -        -
-       -        -         -     -     -
      # region    1338      4   object        -         -       -       -        -
-       -        -         -     -     -
      # charges   1338   1337  float64  1121.87  63770.43 4740.29 9382.03  16639.91
13270.42 9382.03  1639.56  12110.01  1.52  1.61
```

If you wrote a version of this function that “looked right” but produced repeated numeric values for categorical columns, you just encountered one of the most common logical bugs in automation: **values from a prior loop iteration were reused**.

This bug happens when you only assign numeric-stat variables inside the numeric branch and never reset them in the categorical branch. The loop keeps the old values in memory, so categorical columns accidentally inherit numeric values from the prior numeric column.

A reliable fix is exactly what you see above: set placeholders for every branch-specific variable at the start of each loop iteration, and then overwrite them only when the column is numeric.

If you'd like to compare approaches, here are two classic options for handling categorical columns (both are valid). The first "zeros out" variables in an else branch. The second writes placeholders directly to the output table.

```
# Option 1: Use else to reset non-applicable values

def unistats(df):
  import pandas as pd

  output_df = pd.DataFrame(columns=[
    'Count', 'Unique', 'Type',
    'Min', 'Max', '25%', '50%', '75%',
    'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
  ])

  for col in df.columns:
    count = df[col].count()
    unique = df[col].nunique()
    dtype = str(df[col].dtype)

    if pd.api.types.is_numeric_dtype(df[col]):
      min_val = round(df[col].min(), 2)
      max_val = round(df[col].max(), 2)
      q1 = round(df[col].quantile(0.25), 2)
      q2 = round(df[col].quantile(0.50), 2)
      q3 = round(df[col].quantile(0.75), 2)
      mean_val = round(df[col].mean(), 2)
      median_val = round(df[col].median(), 2)
      mode_series = df[col].mode()
      mode_val = round(mode_series.values[0], 2) if len(mode_series) &gt; 0 else
'-'

      std_val = round(df[col].std(), 2)
      skew_val = round(df[col].skew(), 2)
      kurt_val = round(df[col].kurt(), 2)

    else:
      min_val = '-'
      max_val = '-'
      q1 = '-'
      q2 = '-'
      q3 = '-'
      mean_val = '-'
      median_val = '-'
      mode_val = '-'
      std_val = '-'
      skew_val = '-'
      kurt_val = '-'

    output_df.loc[col] = (
      count, unique, dtype,
      min_val, max_val, q1, q2, q3,
      mean_val, median_val, mode_val, std_val, skew_val, kurt_val
    )
```

```
  return output_df
```

```
# Option 2: Write placeholders directly to the output table

def unistats(df):
  import pandas as pd

  output_df = pd.DataFrame(columns=[
    'Count', 'Unique', 'Type',
    'Min', 'Max', '25%', '50%', '75%',
    'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
  ])

  for col in df.columns:
    count = df[col].count()
    unique = df[col].nunique()
    dtype = str(df[col].dtype)

    if pd.api.types.is_numeric_dtype(df[col]):
      min_val = round(df[col].min(), 2)
      max_val = round(df[col].max(), 2)
      q1 = round(df[col].quantile(0.25), 2)
      q2 = round(df[col].quantile(0.50), 2)
      q3 = round(df[col].quantile(0.75), 2)
      mean_val = round(df[col].mean(), 2)
      median_val = round(df[col].median(), 2)
      mode_series = df[col].mode()
      mode_val = round(mode_series.values[0], 2) if len(mode_series) &gt; 0 else
'-'
      std_val = round(df[col].std(), 2)
      skew_val = round(df[col].skew(), 2)
      kurt_val = round(df[col].kurt(), 2)

      output_df.loc[col] = (
        count, unique, dtype,
        min_val, max_val, q1, q2, q3,
        mean_val, median_val, mode_val, std_val, skew_val, kurt_val
      )

    else:
      output_df.loc[col] = (count, unique, dtype, '-', '-', '-', '-', '-', '-', '-
', '-', '-', '-', '-')

  return output_df

unistats(df)

# Output:
#          Count Unique     Type      Min       Max     25%     50%      75%
Mean  Median     Mode       Std  Skew  Kurt
# age       1338     47    int64       18        64    27.0    39.0     51.0
39.21    39.0       18     14.05  0.06 -1.25
# sex       1338      2   object        -         -       -       -        -
-       -        -         -     -     -
# bmi       1338    548  float64    15.96     53.13    26.3    30.4    34.69
30.66    30.4     32.3       6.1  0.28 -0.05
# children  1338      6    int64        0         5     0.0     1.0      2.0
1.09     1.0        0      1.21  0.94   0.2
```

```
        # smoker    1338      2   object        -         -       -       -        -
-       -        -         -     -     -
        # region    1338      4   object        -         -       -       -        -
-       -        -         -     -     -
        # charges   1338   1337  float64  1121.87  63770.43 4740.29 9382.03  16639.91
13270.42 9382.03  1639.56  12110.01  1.52  1.61
```

6.4Automating Visualizations

The stats table gives you the numbers, but distributions are faster to evaluate visually. A histogram paired with a boxplot reveals skewness, spread, and potential outliers in a single glance. For categorical columns, a count plot shows relative group sizes without forcing you to scan a frequency table.

Rather than dumping all the visualization code into unistats(), we will build each chart type as its own function, then combine them with a dispatcher. This is the same modular pattern you will use in the bivariate chapter: small, testable pieces composed into a larger workflow. Each function does one job well.

A. histogram()

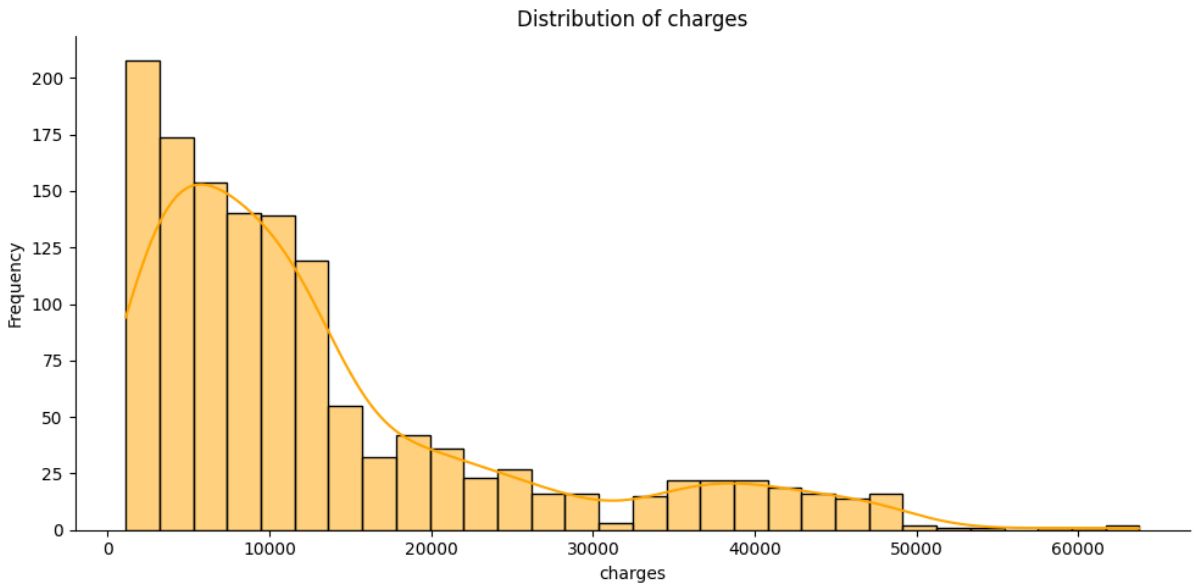
A histogram shows how numeric values are distributed across bins. Overlaying a kernel density estimate (KDE) smooths the shape, making it easier to spot skewness and multimodality. The function below wraps this into a single call.

```
def histogram(df, col):
  import matplotlib.pyplot as plt
  import seaborn as sns
  plt.figure(figsize=(10, 5))
  sns.histplot(data=df, x=col, kde=True, color=&quot;orange&quot;)
  plt.title(f'Distribution of {col}')
  plt.ylabel('Frequency')
  plt.xlabel(col)
  sns.despine()
```

```
plt.tight_layout()
plt.show()
```

The function creates a single figure, draws the histogram with a KDE curve, labels the axes, and removes the top and right spines for a cleaner look. It calls plt.show() at the end so the chart renders immediately.

```
histogram(df, 'charges')
```



Save histogram() to your functions.py file before moving on.

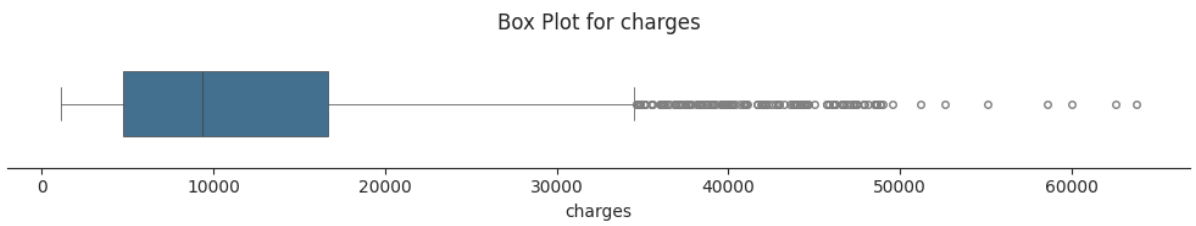
B. boxplot()

A box plot summarizes the five-number summary (min, Q1, median, Q3, max) and flags outliers beyond the whiskers. A compact horizontal layout keeps the chart from dominating the screen when you are reviewing many columns.

```
def boxplot(df, col):
  import matplotlib.pyplot as plt
  import seaborn as sns
  plt.figure(figsize=(10, 2))
  sns.set_style('ticks')
  flierprops = dict(marker='o', markersize=4, markerfacecolor='none',
                    linestyle='none', markeredgecolor='gray')
  sns.boxplot(data=df, x=col, fliersize=4, saturation=0.50,
              width=0.50, linewidth=0.5, flierprops=flierprops)
  plt.title(f'Box Plot for {col}')
  plt.yticks([])
  sns.despine(left=True)
  plt.tight_layout()
  plt.show()
```

The height is deliberately short (figsize=(10, 2)) so the boxplot reads as a thin strip. The flierprops dictionary styles outlier markers as hollow gray circles, keeping them visible without overwhelming the plot. Removing the left spine and y-ticks eliminates visual clutter since the y-axis carries no information for a single horizontal box.

```
boxplot(df, 'charges')
```



Save boxplot() to your functions.py file.

C. countplot()

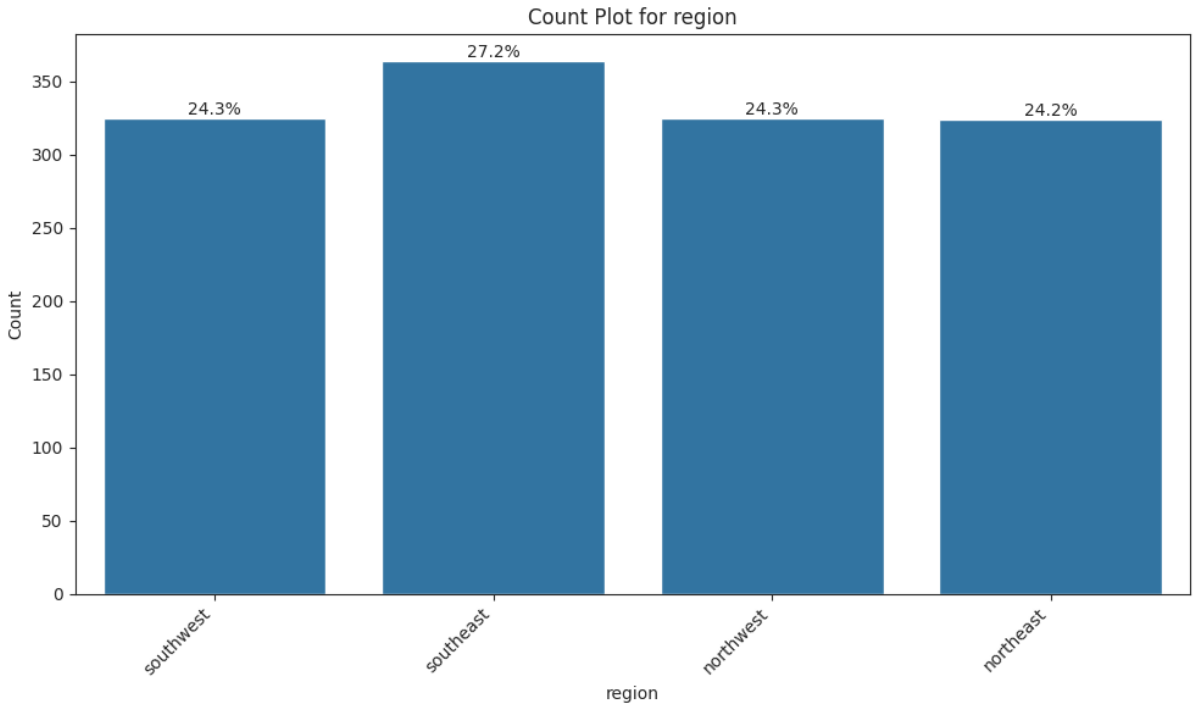
A count plot shows how many rows fall into each category. Adding percentage labels on top of each bar lets you compare relative sizes without mental math. This function handles both true categorical columns and

boolean 0/1 columns (which are technically numeric but behave like categories).

```
def countplot(df, col):
  import matplotlib.pyplot as plt
  import seaborn as sns
  plt.figure(figsize=(10, 6))
  ax = sns.countplot(data=df, x=col)
  plt.title(f'Count Plot for {col}')
  plt.xlabel(col)
  plt.ylabel('Count')
  plt.xticks(rotation=45, ha='right')
  total = len(df[col].dropna())
  for p in ax.patches:
    height = p.get_height()
    percentage = (height / total) * 100
    ax.text(p.get_x() + p.get_width() / 2., height,
            f'{percentage:.1f}%', ha='center', va='bottom')
  plt.tight_layout()
  plt.show()
```

The loop over ax.patches is the key trick. Each bar is a Rectangle patch; we read its height, compute the percentage relative to the non-null total, and place the label just above the bar. The 45-degree rotation on x-tick labels prevents overlap when category names are long.

```
countplot(df, 'region')
```



Save countplot() to your functions.py file.

D. univariate_viz() — The Dispatcher

You now have three standalone chart functions. The next step is a dispatcher that picks the right one automatically based on the column's data type. For non-boolean numeric columns, the default is a stacked layout with the boxplot on top and histogram underneath. For boolean 0/1 columns and categorical columns, it calls countplot(). This is the same pattern you will see in the bivariate chapter, where a dispatcher routes to scatterplot(), bar_chart(), or crosstab() depending on the variable types.

```
def univariate_viz(df, col, stacked=True):
  import pandas as pd
  import matplotlib.pyplot as plt
  import seaborn as sns
```

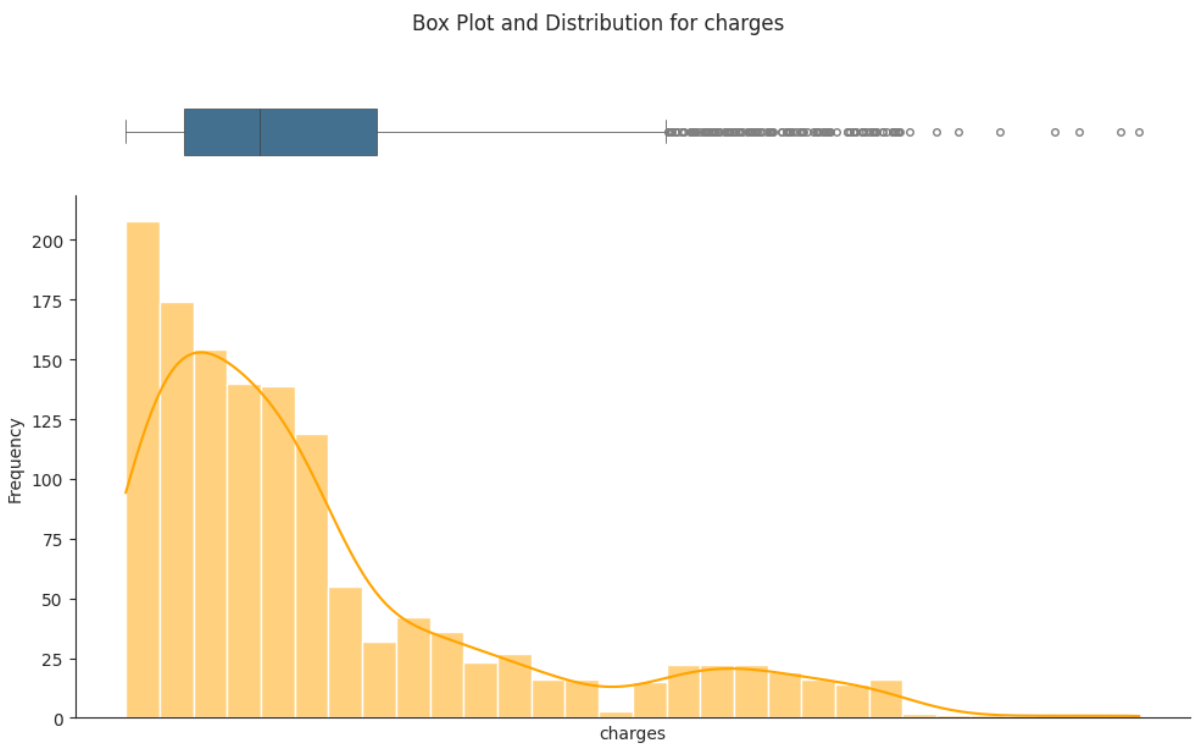
```
        if pd.api.types.is_numeric_dtype(df[col]):
          unique_vals = set(df[col].dropna().unique())
          is_boolean = unique_vals.issubset({0, 1})

          if not is_boolean:
            if stacked:
              f, (ax_box, ax_hist) = plt.subplots(
                2, sharex=True, figsize=(10, 6),
                gridspec_kw={&quot;height_ratios&quot;: (.15, .85)}
              )
              sns.set_style('ticks')
              flierprops = dict(marker='o', markersize=4, markerfacecolor='none',
                                linestyle='none', markeredgecolor='gray')
              sns.boxplot(data=df, x=col, ax=ax_box, fliersize=4, saturation=0.50,
                          width=0.50, linewidth=0.5, flierprops=flierprops)
              sns.histplot(data=df, x=col, ax=ax_hist, kde=True,
color=&quot;orange&quot;)
              ax_box.set(yticks=[], xticks=[])
              ax_box.set_xlabel('')
              ax_box.set_ylabel('')
              ax_hist.set_ylabel('Frequency')
              ax_hist.set_xlabel(col)
              plt.suptitle(f'Box Plot and Distribution for {col}', y=1.02)
              sns.despine(ax=ax_hist)
              sns.despine(ax=ax_box, left=True, bottom=True)
              plt.tight_layout()
              plt.show()
            else:
              boxplot(df, col)
              histogram(df, col)
          else:
            countplot(df, col)
        else:
          countplot(df, col)
```

The stacked parameter controls the numeric layout. When True (the default), the function creates a two-row subplot with the boxplot taking 15% of the height and the histogram taking 85%. When False, it simply calls boxplot() and histogram() as separate figures. Boolean detection uses the same check as before: if the only non-null values are 0 and 1, the column is treated as categorical.

```
# Numeric column: stacked boxplot + histogram
univariate_viz(df, 'charges')

# Categorical column: count plot
univariate_viz(df, 'region')
```



Save univariate_viz() to your functions.py file. Place it below the three individual chart functions and above unistats(), since it calls them.

6.5Integration

You now have four visualization functions and a stats-only unistats(). The final step is connecting them. The pattern is delegation: unistats() handles statistics, univariate_viz() handles charts, and a single parameter—viz—connects them. This is the same architecture you will see in the bivariate chapter, where bivariate() delegates charting to scatterplot(), bar_chart(), and crosstab().

```
def unistats(df, viz=True):
  import pandas as pd

  output_df = pd.DataFrame(columns=[
    'Count', 'Unique', 'Type',
```

```
    'Min', 'Max', '25%', '50%', '75%',
    'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
  ])

  for col in df.columns:
    count = df[col].count()
    unique = df[col].nunique()
    dtype = str(df[col].dtype)
    min_val = '-'
    max_val = '-'
    q1 = '-'
    q2 = '-'
    q3 = '-'
    mean_val = '-'
    median_val = '-'
    mode_val = '-'
    std_val = '-'
    skew_val = '-'
    kurt_val = '-'

    if pd.api.types.is_numeric_dtype(df[col]):
      min_val = round(df[col].min(), 2)
      max_val = round(df[col].max(), 2)
      q1 = round(df[col].quantile(0.25), 2)
      q2 = round(df[col].quantile(0.50), 2)
      q3 = round(df[col].quantile(0.75), 2)
      mean_val = round(df[col].mean(), 2)
      median_val = round(df[col].median(), 2)
      mode_series = df[col].mode()
      mode_val = round(mode_series.values[0], 2) if len(mode_series) &gt; 0 else '-'
      std_val = round(df[col].std(), 2)
      skew_val = round(df[col].skew(), 2)
      kurt_val = round(df[col].kurt(), 2)

    if viz:
      univariate_viz(df, col)

    output_df.loc[col] = (
      count, unique, dtype,
      min_val, max_val, q1, q2, q3,
      mean_val, median_val, mode_val, std_val, skew_val, kurt_val
    )

  return output_df
```

Compare this to the stats-only version from the previous section. Two things changed: a viz parameter (defaulting to True) and one line inside the loop—if viz: univariate_viz(df, col). That single line delegates all charting decisions to the dispatcher you built in the prior section. unistats() does not know how to draw charts; it trusts univariate_viz() to pick the right one.

```
# Default: stats + charts
unistats(df)
```

```
# Table only, no charts
unistats(df, viz=False)
```

Update your functions.py with this version of unistats(). Make sure histogram(), boxplot(), countplot(), and univariate_viz() are also in the file, placed above unistats() so they are defined before they are called.

6.6Reading Univariate Output

You now have a working unistats() function. The real skill is knowing what the output tells you. A stats table full of numbers is only useful if you can translate it into a plan: what to keep, what to transform, what to drop, what to investigate further. The two datasets below illustrate how.

Insurance
You have been using the insurance dataset throughout this chapter. Run unistats(df) one more time and look at the output with data preparation in mind.

What you see	Evidence	What it means for data preparation
charges is heavily right-skewed	Skewness = 1.52, long right tail in histogram	Needs a skew-correcting transformation (log, square root, or Yeo-Johnson)
charges has high-value outliers	Max = 63,770 vs. median = 9,382; boxplot flags points beyond the upper whisker	Outlier handling: cap, remove, or let the transformation address it
children is count data	Only 6 unique values (0–5), skewness = 0.94	Consider treating as categorical rather than continuous
smoker and sex are	2 unique values each, type = object	Encode as 0/1

binary		
region has 4 balanced categories	Countplot shows roughly equal splits	One-hot encode; no binning needed
No missing data	Count = 1,338 for every column	No imputation or row-dropping required
No ID-like columns	No column where Unique ≈ Count	Nothing to drop on structural grounds

Insurance is a clean dataset. The main preparation needs are skew correction on charges, encoding the three categorical columns, and a decision about whether children should be treated as categorical. Most real-world data is messier.

Airbnb Listings

The Airbnb listings dataset has 20,025 rows and 16 columns. Load it and run unistats(df_airbnb). The output paints a very different picture.

What you see	Evidence	What it means for data preparation
id is an identifier	Unique = Count (20,025)	Drop. Identifiers carry no predictive value.
host_id is near-unique	17,502 unique out of 20,025	Drop. Too many unique values to be useful as a feature.
neighbourhood_group is entirely empty	Count = 0	Drop. No data to work with.
name and host_name are text	Object type, high uniqueness	Drop for standard tabular analysis
last_review is a date stored as a string	Type = object, values look like dates	Parse into date components (year, month, days since last review)
neighbourhood has high cardinality	Many unique values relative to	Bin rare categories into an “Other” group

	row count	
price is extremely right-skewed	Skewness = 26.83, max = 9,000 vs. median = 130	Skew correction and outlier handling. Likely both.
minimum_nights has extreme outliers	Skewness = 51.77, max = 1,001 vs. median = 2	Outlier removal or capping. A 1,001-night minimum is almost certainly a data error.
average_review and last_review share the same missing pattern	Both have 2,404 nulls (12%)	Listings with no reviews have no review date or rating. Structurally missing, not random.
latitude and longitude are coordinates	Narrow range, low skew, continuous	Not useful as raw numeric features. Drop or engineer into distance-based features.

The Airbnb data needs column removal, date parsing, category binning, skew correction, outlier handling, and missing value management. Insurance needed almost none of that. Same function, different datasets, very different preparation plans.

Here is a checklist you can fill out after running unistats() on any new dataset. Complete it before you start cleaning.

Data Preparation Checklist

- Columns to drop (IDs, empty columns, constant values, free text)
- Dates to parse into structured features
- High-cardinality categoricals to bin
- Skewed numerics to transform
- Missing data patterns to investigate
- Outliers to evaluate

- Encoding needs (binary, multi-category, ordinal)

The next chapter builds the functions that act on these findings. Each item on this checklist maps to a specific cleaning function you will create, test, and save to functions.py.

6.7External Functions File

You now have five functions: histogram(), boxplot(), countplot(), univariate_viz(), and unistats(). Rather than copying this code into every new notebook, save all five in an external Python file called functions.py. Over the next two chapters, you will add more functions to this same file, building a personal data science toolkit one piece at a time. Any notebook can then import the file and call your functions directly.

```
import pandas as pd
import sys

sys.path.append('/content/drive/MyDrive/Colab Notebooks/class/Predictive AI ML/In-
class notebooks')
import functions as fun  # Do not include &quot;.py&quot; in the import

df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
fun.unistats(df)
```

But first you need to create the file. The steps depend on whether you are working in Google Colab, a local IDE like VS Code or Cursor, or something else entirely.

Google Colab

Colab does not have a “create new file” button for your Drive folders. The simplest approach is %%writefile, a built-in Jupyter magic command. In any notebook cell, put %%writefile followed by the full Drive path on the very

first line. Paste all your functions below it. When you run the cell, the file is created on your Drive (or overwritten if it already exists).

```
%%writefile /content/drive/MyDrive/Colab Notebooks/class/Predictive AI ML/In-class notebooks/functions.py

# Paste all five functions below this line:
# histogram, boxplot, countplot, univariate_viz, unistats
```

Once the file exists on your Drive, you can also edit it directly inside Colab. In the left sidebar, click the folder icon to open the file browser and navigate to /content/drive/MyDrive/Colab Notebooks/class/Predictive AI ML/In-class notebooks/. Double-click functions.py to open it in a basic text editor. Edits save with Ctrl+S. In future chapters, when you need to add new functions, you can either edit the file this way or re-run a %%writefile cell with the full updated contents.

VS Code and Cursor

Right-click in the file explorer panel, select *New File*, and name it functions.py. Save it in the same folder as your notebook files. Paste all five functions and save.

Importing is simpler when you work locally. If functions.py sits in the same folder as your notebook or script, you do not need sys.path.append. Just import directly:

```
import functions as fun
fun.unistats(df)
```

Creating the File Manually

If you do not have an IDE, you can create the file with any plain-text editor. On Windows, open Notepad, paste your functions, then go to File > Save As. Change “Save as type” to “All Files (*.*)” and type functions.py as the filename. On Mac, open TextEdit, go to Format > Make Plain Text first, then

paste and save as functions.py. If you are using Google Colab, drag the saved file into the correct folder on Google Drive so the sys.path.append path can find it.

Whichever method you use, the result is the same: a file called functions.py containing all five functions. Every function you build in the next two chapters goes into this same file.

Order matters. The helper functions (histogram, boxplot, countplot) must appear above the dispatcher (univariate_viz), which must appear above unistats). Python reads a file from top to bottom, so a function must be defined before another function can call it.

6.8Practice

Work through these problems to reinforce what you have built:

Practice #1: Extending Visualization Functions
Your visualization functions work, but they lack two features that are useful in practice: saving charts to disk and controlling chart size. Extend histogram(), boxplot(), and countplot() with the following parameters:

- A save_path parameter (string or None, default None). When provided, save the chart as a PNG file using plt.savefig(save_path, bbox_inches='tight', dpi=200). Call savefig() before plt.show().
- A figsize parameter (tuple) that controls the dimensions of each chart. Default to each function's current value: (10, 5) for histogram(), (10, 2) for boxplot(), and (10, 6) for countplot().

Test your extended functions on the insurance.csv dataset. Verify that save_path actually writes files to disk, and that different figsize values

change the chart dimensions as expected.

Practice #2: Categorical Frequency Tables

Your unistats() function reports Count and Unique for categorical columns, but nothing about how the values are distributed across categories. Your count plots show this visually, but sometimes you need the numbers. Create a function called freq_tables(df) that generates a frequency table for each non-numeric column in a DataFrame.

For each non-numeric column, your function should print a table showing:

- Each unique value
- Its count
- Its percentage of the total
- The cumulative percentage (sorted from most to least frequent)

Return a dictionary where each key is a column name and each value is a DataFrame containing that column's frequency table. Skip numeric columns entirely.

Test your function on the insurance.csv dataset. Which region has the most policyholders? Are the categories roughly balanced or heavily skewed toward one value?

The next practice problem is more advanced. You may need to use AI to help you design, debug, and test your solution. If you do, make sure you understand every line of code you submit.

Practice #3: Datetimes in unistats()

Update your **unistats()** function so it can detect and summarize date/time columns. Many real datasets store dates as text (e.g., "2024-09-17") rather than as true datetime values.

Your function should (1) correctly identify datetime columns, including columns that are currently stored as strings, and then (2) add datetime-friendly outputs for those columns.

For each datetime column, add these outputs to the unistats table:

- **MinDate** (earliest date)
- **MaxDate** (latest date)
- **DateRangeDays** (difference in days between max and min)
- **MostCommonYear** (the year that appears most often)

To test your work, create a small synthetic dataset that includes at least one numeric column, one categorical column, and one datetime column stored as strings. Then run **unistats()** and verify that the datetime column produces valid values in the new fields.

6.9Homework

Complete the assessment below:

This assessment can be taken online.